Tech Challenge — Fase 5

Revenda de veículos · Arquitetura de software · Entrega técnica consolidada · 05/11/2026

Arquitetura serverless/gerenciada, segurança dos dados, SAGA e rastreabilidade dos requisitos.

1. Desenho da arquitetura e cinco diagramas AWS
2. Relatório de segurança dos dados
3. Relatório SAGA
4. Matriz de requisitos

Repositório: `github.com/Edufreitass/fase5-tech-challenge-soat` (se privado, o avaliador precisa receber acesso).

> Esta entrega descreve código e infraestrutura declarada. `infra/config/dev.yml` contém `destroy: true`: não há implantação ativa. Testes locais não comprovam latência, custos, achados de segurança ou comportamento E2E na AWS. Gateway Pix simulado.

Desenho da arquitetura

Revenda de veículos · Tech Challenge SOAT · Fase 5 · 05/11/2026

> Esta entrega descreve código e infraestrutura declarada. `infra/config/dev.yml` contém `destroy: true`: não há implantação ativa. Testes locais não comprovam latência, custos, achados de segurança ou comportamento E2E na AWS.

1. Objetivo e atendimento ao enunciado

API de revenda de veículos: cadastro e edição do catálogo, cadastro de compradores, listagens por preço crescente, reserva concorrente, código de pagamento, pagamento, desistência, retirada e baixa. O frontend React é uma entrega adicional. O pagamento é uma simulação Pix, sem integração bancária real; a equipe aprova ou recusa o pagamento pelo painel.

Código: `github.com/Edufreitass/fase5-tech-challenge-soat` (se privado, o avaliador deve receber acesso). PDFs em `docs/deliverables/`; fontes dos PDFs em `docs/deliverables/src/`; diagramas editáveis em `docs/diagrams/`.

2. Serviços e justificativas (serverless e gerenciados)

Serviço	Uso implementado	Razão da escolha
API Gateway REST	Contrato OpenAPI, integração com Lambdas, authorizer Cognito e throttling	Interface gerenciada para o frontend e o Swagger; sem servidor a operar
Lambda (Python 3.13, arm64)	Handlers REST e workers, uma role IAM por função	Paga por uso; arm64 reduz custo; escala sem administração
DynamoDB	Seis tabelas: veículos, clientes, vendas, pagamentos, idempotência e gateway simulado	Escritas condicionais e transações resolvem a concorrência; o GSI com preço na sort key entrega a listagem ascendente
Step Functions Standard	SAGA orquestrada com esperas por task token, timeouts e compensações	Esperas de dias, compensações explícitas e histórico por execução
Cognito	Dois user pools (clientes e funcionários), MFA obrigatório na equipe	Autenticação gerenciada; separa populações de risco distinto
SNS e SQS	Eventos de domínio, fila de auditoria e fila de webhooks do gateway simulado, com DLQ (14 dias)	Desacopla o que não deve bloquear o fluxo principal
DynamoDB Streams	Detecta CRIADO → RESERVADO e inicia a SAGA	O fluxo nasce do dado persistido, sem publicar em dois lugares
EventBridge Scheduler	Varredura de vendas órfãs e rotina diária de retenção LGPD	Agendamento gerenciado, sem servidor de cron
SSM Parameter Store (SecureString)	Segredos e chave privada de criptografia por campo	Segredos fora do código, cifrados com a chave KMS gerenciada pela AWS
CloudWatch e X-Ray	Logs, métricas, 9 alarmes, painel e tracing	Observabilidade nativa, sem agentes

S3 privado + CloudFront	SPA estática com OAC e cabeçalhos de segurança; encaminha /api/* ao API Gateway	Site estático sem servidor; não há VPC provisionada
Terraform + GitHub Actions (OIDC)	Infraestrutura declarativa e credenciais temporárias no pipeline	Ambiente reproduzível; sem chaves de longa duração

3. Consistência e fluxo de compra

A API valida o comprador e grava a venda CRIADO. Em uma segunda transação, reserva o veículo disponível, cria o lock do cliente, muda a venda para RESERVADO e grava o histórico. Uma disputa perdedora recebe 409 e sua venda é cancelada. O Stream observa a transição e inicia uma execução nomeada por vendaId, evitando duplicação.

A SAGA emite a cobrança, espera o pagamento, prepara a retirada e conclui a venda com o veículo VENDIDO. Desistência, expiração e ausência de retirada têm compensações (detalhes no relatório SAGA).

4. Serviços de segurança utilizados

Serviço	Uso	Por que
Cognito	Autenticação em dois pools; MFA obrigatório para funcionários	A aplicação não guarda senhas; o MFA reduz o risco de conta de funcionário comprometida
API Gateway (authorizer e throttling)	Valida o token antes da Lambda e limita a taxa	Rejeita na borda o que não está autenticado e reduz abuso
IAM (role por Lambda + permissions boundary)	Menor privilégio; a chave privada PII é legível em runtime só por clientes-interno e lgpd-worker	Limita o dano se uma função for comprometida
SSM Parameter Store + KMS gerenciado	Segredos e chave privada cifrados em repouso	Segredos fora do código e do repositório
S3 privado + CloudFront (OAC, cabeçalhos)	Hospedagem do frontend	Sem acesso público ao bucket; todo acesso passa pelo CloudFront
HMAC e SigV4	Webhook público valida assinatura e timestamp; rotas internas do gateway simulado usam IAM/SigV4	Impede webhook falso ou repetido
CloudWatch Logs, SNS e SQS	Trilha de auditoria (retenção de 1827 dias) e logs de aplicação (7 dias)	Permite saber quem acessou dados pessoais
OIDC do GitHub Actions	Credenciais temporárias no pipeline	Sem chaves AWS de longa duração

Controles de segurança não implementados (apenas documentados)

Os controles abaixo **não estão implementados** no código nem no Terraform: foram deixados de fora por custo, pois o projeto é acadêmico e roda em conta pessoal. Estão apenas documentados como evolução para produção. Os valores variam por região e mudam com o tempo; confirme nas páginas de preço da AWS.

Controle	O que protegeria	Justificativa de custo	Mitigação atual
AWS WAF	Bloqueio de ataques web na borda	Mensalidade fixa por Web ACL e por regra, mais cobrança por requisição; sem cota gratuita	Throttling no API Gateway, validação de entrada, Cognito, cabeçalhos de segurança
Amazon GuardDuty	Detecção gerenciada de ameaças na conta	Gratuito só durante a avaliação; depois cobra por volume de eventos analisados	Alarmes CloudWatch e auditoria de aplicação
AWS Security Hub	Consolidação de achados e padrões (CIS, FSBP)	Gratuito só na avaliação; depois cobra por verificação e achado, e depende do AWS Config	Revisão de IaC, bandit, pip-audit, gitleaks e zizmor no CI
Amazon Macie	Descoberta de dados sensíveis em S3	Cobra por bucket e por dado inspecionado; dados pessoais não ficam em S3 (só o site estático)	Dados pessoais no DynamoDB, com CPF/RG/CNH cifrados por campo
Chave KMS gerenciada pelo cliente	Política, rotação e auditoria próprias da chave	Mensalidade por chave e cobrança por uso; os segredos usam `alias/aws/ssm` (chave gerenciada pela AWS)	Cifra por campo (libsodium) com chave privada em SSM SecureString, lida só por duas funções
DynamoDB PITR	Restauração a um ponto no tempo	Cobrado por GB-mês de backup contínuo, fora do free tier	Tabelas e dados iniciais recriáveis por Terraform e migrações; vendas reais não são recuperáveis
CloudTrail próprio	Auditoria permanente das chamadas à API AWS	Trail adicional exige armazenamento em S3 e, para eventos de dados, cobrança por evento	Histórico padrão da conta (90 dias) e auditoria de aplicação (SNS → SQS → CloudWatch Logs)

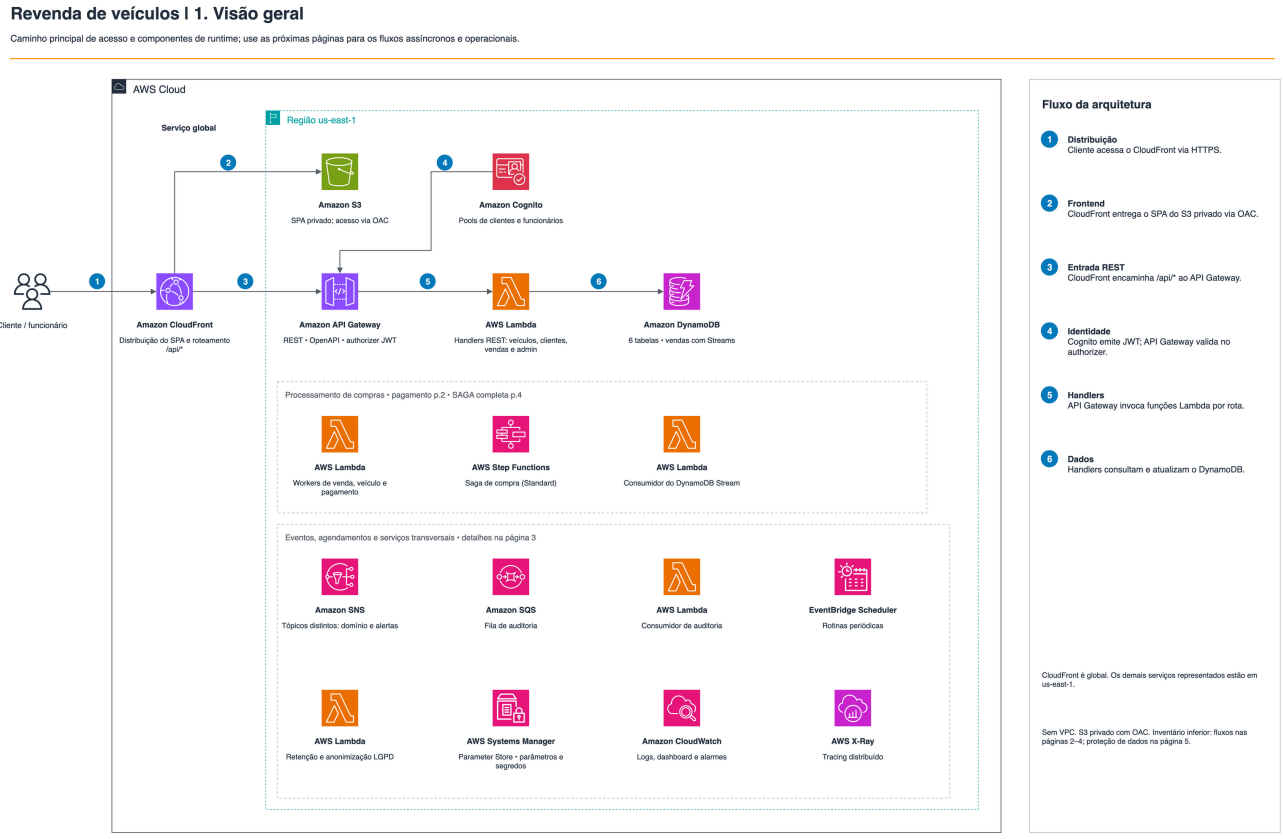
5. Custo e limites operacionais

O desenho prioriza serviços gerenciados e redução de operação, não gratuidade garantida. Franquias, créditos e elegibilidade da conta não foram verificados. DynamoDB provisionado, transições da SAGA e observabilidade podem gerar cobrança; o Budget criado no bootstrap é um alerta, não um limite. O gateway é simulado e a documentação de retirada não integra órgão de trânsito.

6. Convenções dos diagramas

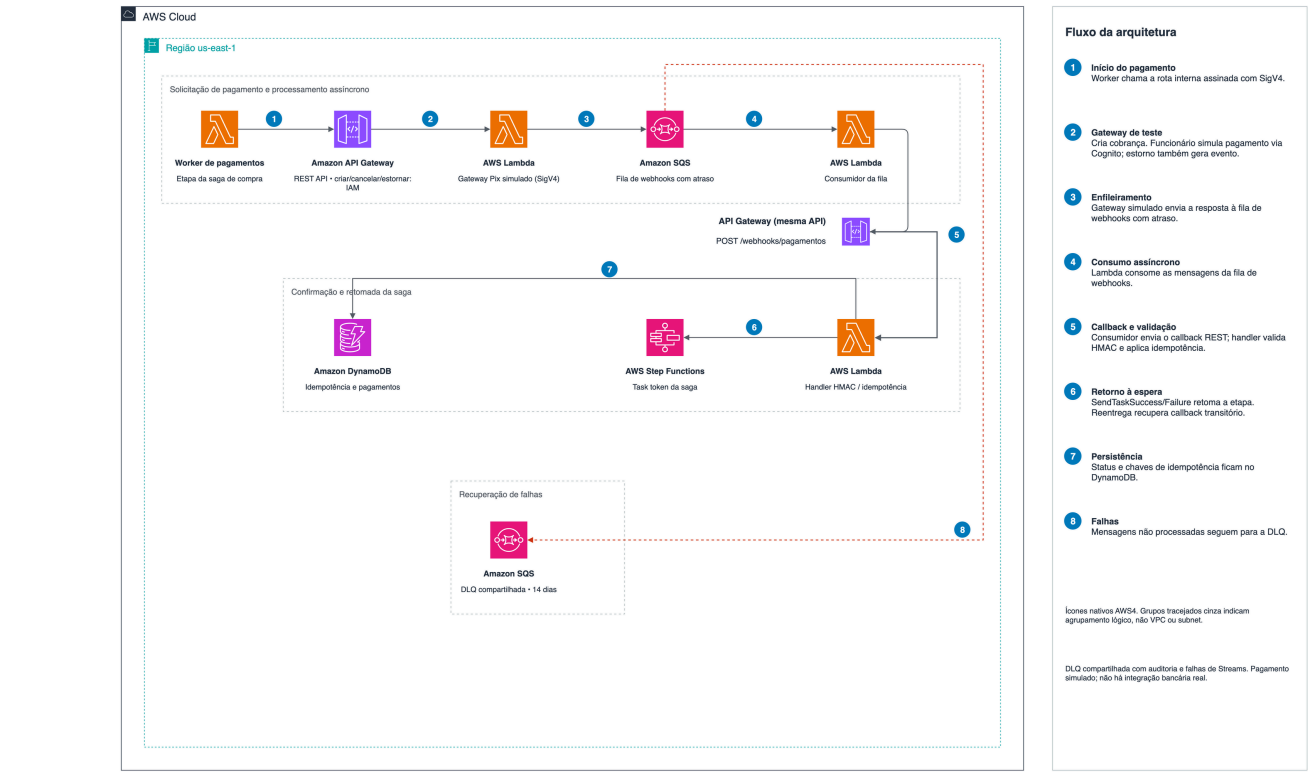
Fonte editável: `docs/diagrams/arquitetura-aws-revenda.drawio`, com ícones AWS4 nativos. Não se representam VPC ou subnets inexistentes. Ícones repetidos do API Gateway representam a mesma API em etapas diferentes.

Diagrama 1 — Visão geral



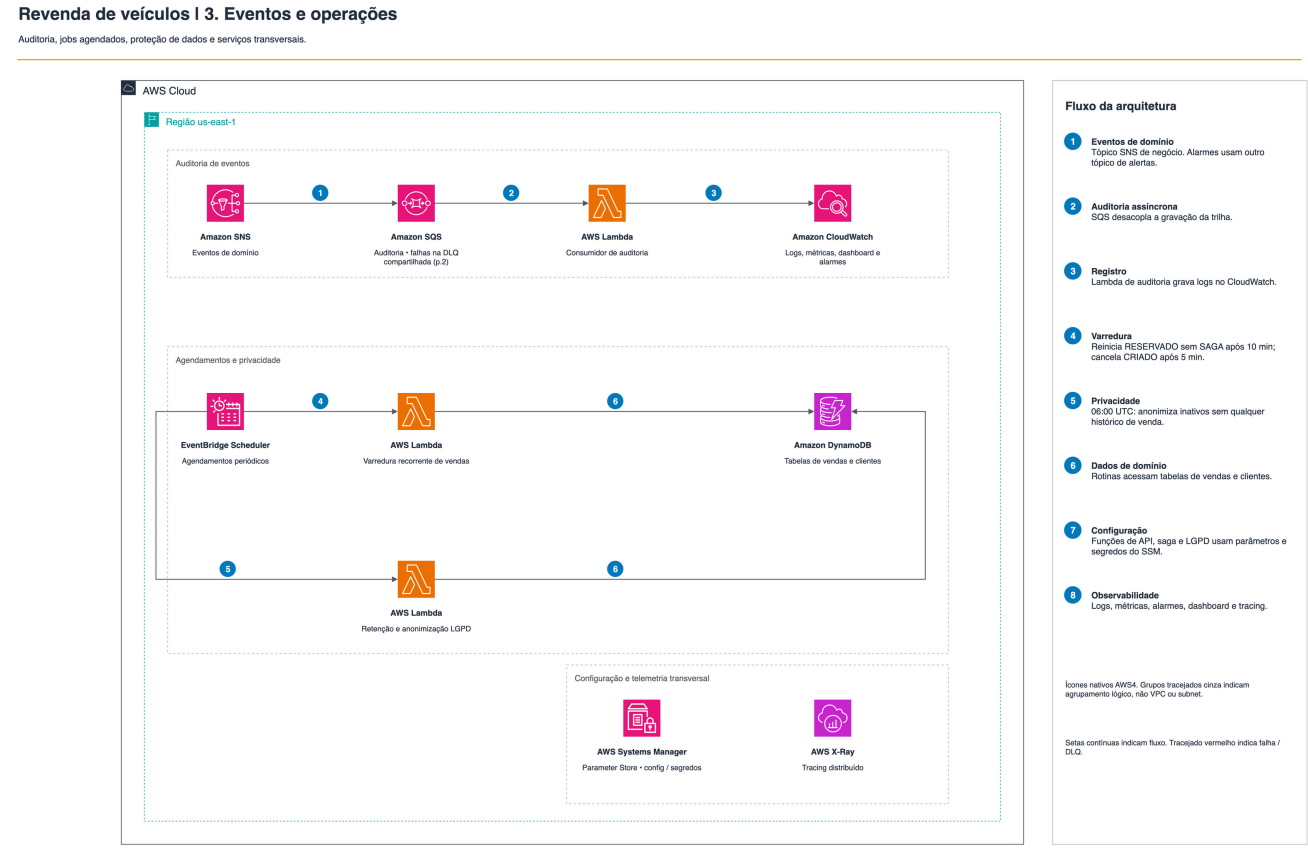
Fonte editável: arquitetura-aws-revenda.drawio, página 1. Serviços declarados; implantação não verificada.

Diagrama 2 — Pagamento e callback



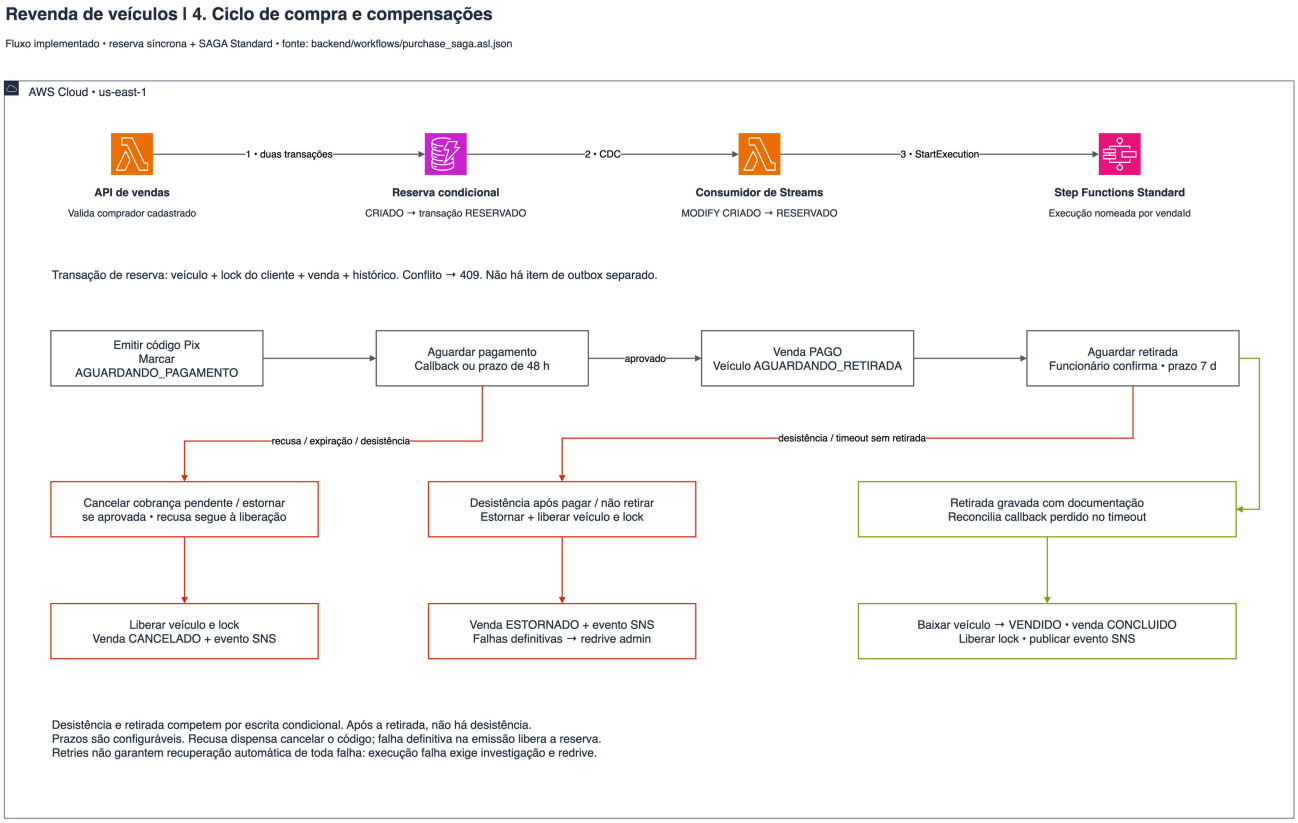
Fonte editável: arquitetura-aws-revenda.drawio, página 2. Serviços declarados; implantação não verificada.

Diagrama 3 — Eventos e operações



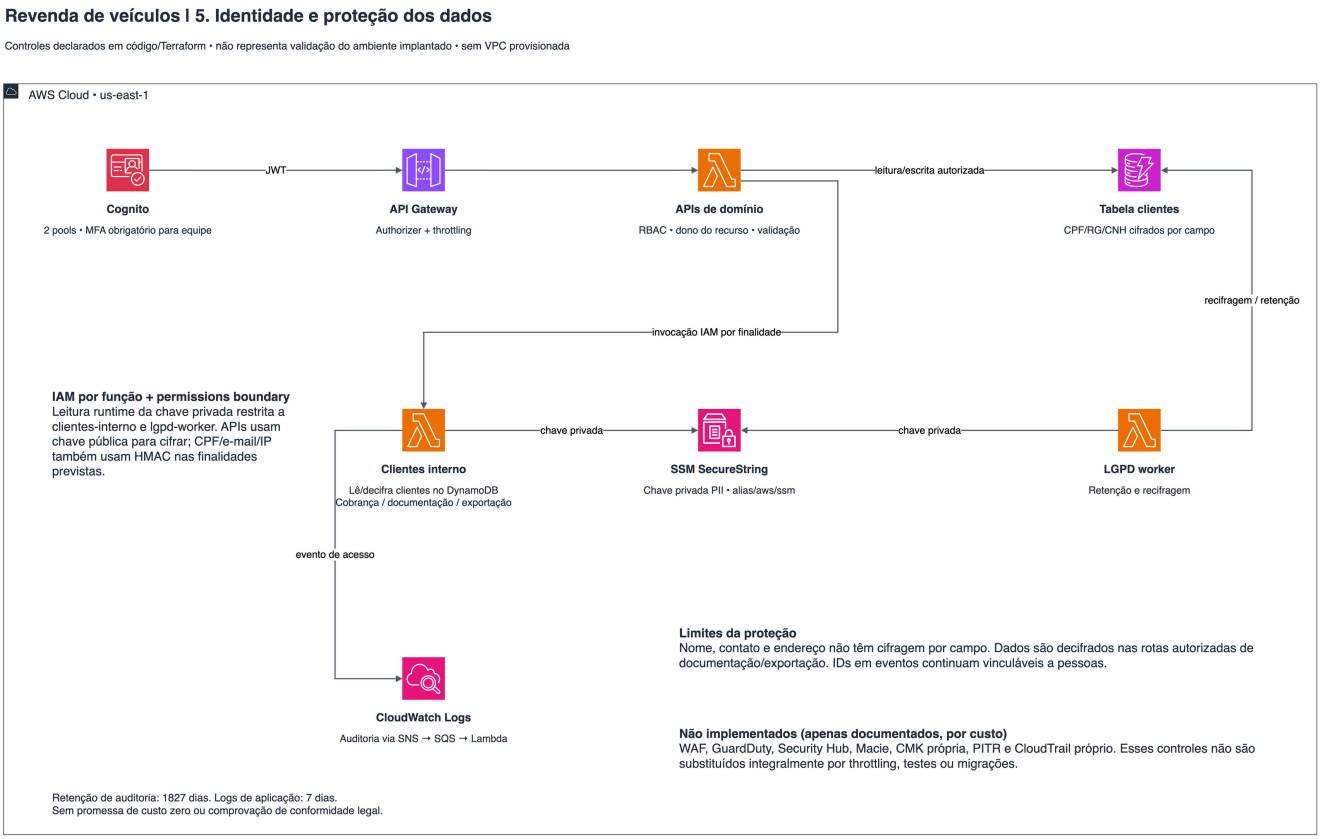
Fonte editável: arquitetura-aws-revenda.drawio, página 3. Serviços declarados; implantação não verificada.

Diagrama 4 — Ciclo completo e compensações



Fonte editável: arquitetura-aws-revenda.drawio, página 4. Serviços declarados; implantação não verificada.

Diagrama 5 — Identidade e proteção de dados



Fonte editável: arquitetura-aws-revenda.drawio, página 5. Serviços declarados; implantação não verificada.

Relatório de segurança dos dados

Revenda de veículos · Tech Challenge SOAT · Fase 5 · 05/11/2026

> Esta entrega descreve código e infraestrutura declarada. `infra/config/dev.yml` contém `destroy: true`: não há implantação ativa. Testes locais não comprovam latência, custos, achados de segurança ou comportamento E2E na AWS.

1. Inventário dos dados

Dados / localização	Natureza e proteção
Catálogo (DynamoDB veículos)	Marca, modelo, ano, cor, preço e disponibilidade; listagem pública dos disponíveis. Placa, chassi e RENAVAM e a gestão ficam restritos a funcionários.
Perfil (DynamoDB clientes)	Nome, e-mail, telefone e endereço: sem cifragem por campo; protegidos por IAM, autorização e criptografia em repouso. Não devem aparecer em logs.
CPF / RG / CNH	Cifra por campo (sealed box X25519/libsodium). CPF tem também HMAC para unicidade; e-mail e IP de consentimento têm HMAC nos usos previstos. CNH opcional; cadastro completo exigido na compra.
Vendas e pagamentos	IDs vinculáveis ao titular, snapshot do veículo, valor, estados, histórico, código Pix e referências do gateway. Nenhum cartão.
Consentimentos e solicitações	Versões e finalidades, datas, IP protegido por HMAC e registro das solicitações LGPD.
Credenciais e chaves	Senhas gerenciadas pelo Cognito. Tokens no sessionStorage do navegador. Segredos e chave privada no SSM SecureString (KMS gerenciado pela AWS).
Auditoria e telemetria	Eventos com IDs, ator, operação e finalidade (potencialmente pessoais por vínculo). Logs de aplicação: 7 dias; auditoria: 1827 dias.

CPF, RG e CNH são tratados como dados pessoais de alto risco de fraude. A qualificação jurídica e as bases legais devem ser validadas antes de uso real.

2. Políticas de acesso

O API Gateway valida o Cognito nas rotas protegidas. Os handlers verificam token de acesso, client ID, emissor, papel e dono do recurso. Admin herda vendedor; clientes não acessam vendas de terceiros. Testes exercitam a matriz de rotas × perfis e cenários de IDOR.

Uma role IAM por Lambda e uma permissions boundary limitam o acesso. Em runtime, só clientes-interno e lgpd-worker leem a chave privada PII. Respostas comuns mascaram documentos; a função interna decifra para cobrança, documentação autorizada e exportação do titular.

3. Tratamento e retenção

Consentimentos versionados no cadastro. Exportação do titular autenticada, limitada a uma por 24 h. Exclusão recusada durante compra em andamento; fora disso, anonimiza o perfil e remove o login Cognito. Com vendas concluídas, preserva registro mínimo (nome, CPF cifrado e IDs de vendas) em RETENCAO_LEGAL com TTL de cerca de cinco anos. É política do exercício, não validação do prazo legal. Rotina diária (06:00 UTC) anonimiza inativos há 730 dias, sem histórico de venda.

4. Segurança operacional

MFA obrigatório para funcionários, autorização de negócio, TLS, cabeçalhos de segurança, sanitização de logs. Webhook com HMAC, timestamp, validação de valor e idempotência; rotas internas do gateway simulado usam SigV4.

Primeiro administrador. Nenhuma conta administrativa é criada por script. A rota pública `POST /configuracao-inicial` só funciona enquanto o grupo admin está vazio e só convida o e-mail declarado na implantação; a senha temporária vai só à caixa do dono. Responde 409 depois do primeiro, tem limite de taxa e é auditada.

Terraform e GitHub Actions/OIDC permitem revisão e credenciais temporárias. O CI executa SAST (bandit), auditoria de dependências, análise de workflows e detecção de segredos (gitleaks). Isso não prova ausência de vulnerabilidades. Em incidente: identificar e conter, preservar evidências, corrigir a causa, avaliar obrigações com os responsáveis e verificar a recuperação.

5. Riscos e mitigação

Risco	Mitigação implementada	Limite residual
Exfiltração de clientes	Cifra CPF/RG/CNH, IAM/boundary, controles de dono e papel	Nome, contato e endereço legíveis a principais autorizados; comprometer a função interna expõe dados
Reserva ou retirada concorrente	Transações, condições e lock; desistência persistida antes do callback	Sem transação ACID global; falhas entre serviços exigem compensação
Webhook falso ou repetido	HMAC, timestamp, validação de valor, idempotência	Segredo comprometido permite falsificação
Callback perdido	Reentrega e reconciliação no timeout	Falha definitiva pode exigir redrive administrativo
Perda de dados	Estrutura e seeds reproduzíveis por IaC e migrações	Sem PITR (custo): vendas reais não são recuperáveis; definir backup antes de produção
Abuso / XSS / credenciais	Throttling, validação, CSP, MFA, revogação de sessão	Sem WAF (custo); sessionStorage não protege contra XSS na origem
Criação indevida do primeiro administrador	Rota só convida o e-mail declarado na implantação; fecha após o primeiro; limite de taxa e auditoria	Quem controlar a caixa de e-mail do dono assume a administração
Observabilidade insuficiente	Logs, auditoria, métricas e 9 alarmes	Sem GuardDuty/Security Hub (custo): sem detecção gerenciada de ameaças
Retenção excessiva	Anonimização, TTL do registro mínimo, rotina de inativos	Cobertura parcial por categoria; exige revisão de base legal

6. Controles de segurança não implementados (apenas documentados)

Os controles abaixo **não estão implementados** no código nem no Terraform: foram deixados de fora por custo, pois o projeto é acadêmico e roda em conta pessoal. Estão apenas documentados como evolução para produção. Os valores variam por região e mudam com o tempo; confirme nas páginas de preço da AWS.

Controle	O que protegeria	Justificativa de custo	Mitigação atual
AWS WAF	Bloqueio de ataques web na borda	Mensalidade fixa por Web ACL e por regra, mais cobrança por requisição; sem cota gratuita	Throttling no API Gateway, validação de entrada, Cognito, cabeçalhos de segurança
Amazon GuardDuty	Detecção gerenciada de ameaças na conta	Gratuito só durante a avaliação; depois cobra por volume de eventos analisados	Alarmes CloudWatch e auditoria de aplicação
AWS Security Hub	Consolidação de achados e padrões (CIS, FSBP)	Gratuito só na avaliação; depois cobra por verificação e achado, e depende do AWS Config	Revisão de IaC, bandit, pip-audit, gitleaks e zizmor no CI
Amazon Macie	Descoberta de dados sensíveis em S3	Cobra por bucket e por dado inspecionado; dados pessoais não ficam em S3 (só o site estático)	Dados pessoais no DynamoDB, com CPF/RG/CNH cifrados por campo
Chave KMS gerenciada pelo cliente	Política, rotação e auditoria próprias da chave	Mensalidade por chave e cobrança por uso; os segredos usam `alias/aws/ssm` (chave gerenciada pela AWS)	Cifra por campo (libsodium) com chave privada em SSM SecureString, lida só por duas funções
DynamoDB PITR	Restauração a um ponto no tempo	Cobrado por GB-mês de backup contínuo, fora do free tier	Tabelas e dados iniciais recriáveis por Terraform e migrações; vendas reais não são recuperáveis
CloudTrail próprio	Auditoria permanente das chamadas à API AWS	Trail adicional exige armazenamento em S3 e, para eventos de dados, cobrança por evento	Histórico padrão da conta (90 dias) e auditoria de aplicação (SNS → SQS → CloudWatch Logs)

7. Evidências e limites

Testes automatizados de autorização, criptografia, privacidade, SAGA e contratos rodam localmente (555 testes do backend, cobertura de 95%). Não houve varredura de logs implantados, auditoria jurídica ou avaliação de achados na conta AWS.

Relatório SAGA

Revenda de veículos · Tech Challenge SOAT · Fase 5 · 05/11/2026

> Esta entrega descreve código e infraestrutura declarada. `infra/config/dev.yml` contém `destroy: true`: não há implantação ativa. Testes locais não comprovam latência, custos, achados de segurança ou comportamento E2E na AWS.

1. Tipo escolhido e justificativa

SAGA orquestrada em AWS Step Functions Standard. Vendas coordena passos de Veículos e Pagamentos; Clientes fornece validação e dados. A definição está em `infra/workflows/purchase_saga.asl.json` (34 estados).

O fluxo tem sequência definida, compensações diferentes antes e depois do pagamento e esperas longas. O orquestrador torna essas decisões explícitas e dá histórico por venda. A coreografia reduziria a dependência de um coordenador, mas espalharia prazos e compensações entre consumidores, dificultando saber em que passo uma venda está e por quê. SNS/SQS continuam úteis para auditoria assíncrona, fora do caminho crítico.

2. Limite entre reserva e orquestração

A reserva ocorre antes do StartExecution: a API grava a venda CRIADO e, em transação DynamoDB, condiciona a disponibilidade/versão do veículo, cria o lock do cliente e muda a venda para RESERVADO. O Stream inicia a execução, nomeada por vendaId. Uma varredura inicia RESERVADO sem SAGA após dez minutos e cancela CRIADO antigo após cinco.

3. Caminho de sucesso

Etapa	Efeito
EmitirCodigoPagamento	Worker consulta clientes-interno, cria cobrança idempotente no gateway simulado e persiste o pagamento.
AguardarPagamento	Venda AGUARDANDO_PAGAMENTO; registra task token e espera até o prazo (padrão 48 h).
Webhook / callback	Gateway → SQS → consumidor → API Gateway → webhook HMAC; atualiza o pagamento e envia SendTaskSuccess/Failure. Reentrega não duplica a venda.
MarcarVendaPaga	Venda PAGO, veículo AGUARDANDO_RETIRADA, define prazo e publica confirmação.
AguardarRetirada	Vendedor/admin registra horário e protocolo da documentação; espera padrão de sete dias.
BaixarVeiculo / ConcluirVenda	Veículo VENDIDO e venda CONCLUIDO; libera o lock e publica evento. A retirada registrada é o ponto após o qual a desistência é rejeitada.

4. Exceções e compensações

Cenário	Caminho implementado

Dois compradores / reserva ativa	Transação condicional escolhe um vencedor; o perdedor recebe 409 e sua venda CRIADO é cancelada.
Falha definitiva ao emitir o código	Após retries, libera o veículo e cancela a venda.
Recusa de pagamento	Libera o veículo, cancela a venda, libera o lock e publica.
Expiração ou desistência antes de PAGO	Fecha condicionalmente o pagamento pendente antes de chamar o gateway; se a aprovação venceu a corrida, estorna antes de liberar e cancelar.
Pagamento depois de cancelado/expirado	Marca APROVADO_TARDIO e solicita estorno assíncrono e idempotente.
Desistência após pagar	Persistir a desistência impede a retirada concorrente; a SAGA estorna, libera o veículo, marca ESTORNADO e libera o lock.
Timeout da retirada	ResolverRetirada fecha a janela: se retiradaEm já existe, conclui; senão, estorna e libera.
Falha temporária no callback	Repetir a confirmação com os mesmos dados reenvia o callback (202); dados diferentes não substituem uma retirada registrada.
Falhas definitivas em outros passos	Podem terminar em falha; investigar e usar redrive administrativo. Não há promessa de rollback automático.

5. Idempotência e limites distribuídos

Não existe commit ACID entre DynamoDB, Lambda, gateway e Step Functions. A idempotência vem de referências determinísticas, condições e reexecução dos passos. Tasks usam timeout e retries exponenciais com jitter (seis tentativas). O cliente HTTP do gateway tem timeout de cinco segundos e circuit breaker em memória por instância (não distribuído). A entrada da SAGA contém IDs, valor, correlação e prazos, sem CPF, nome, e-mail ou código Pix.

6. Verificação

Testes locais cobrem reserva concorrente, transições, callbacks, recusa, expiração, estorno, desistência e retirada, incluindo regressões de corrida. O diagrama da compra está na página 4 do arquivo `.drawio`. Resultados locais não são atribuídos a um ambiente AWS publicado.

Matriz de requisitos

Caminhos relativos à raiz do projeto.

Requisito do enunciado	Implementação verificável	Evidência / limite
API para o frontend	API Gateway REST + Lambdas; contrato `backend/contracts/openapi.json`	`backend/tests/contract` ; frontend adicional em `frontend/`
Cadastrar e editar veículos (marca, modelo, ano, cor, preço)	`backend/src/vehicles/` ; POST/PATCH /veiculos	`backend/tests/unit/vehicles/` ; edição com controle de versão
Listar disponíveis e vendidos por preço crescente	GSI `gsi_status_preco` , leitura ascendente	Testes do catálogo; vendidos acessíveis a funcionários
Cadastro de compradores; só cadastrados compram	`backend/src/customers/` ; validação em `sales/application/purchase.py`	Cadastro inexistente ou incompleto é rejeitado antes da reserva
Selecionar e reservar	IniciarCompra: venda CRIADO; transação reserva veículo, lock e RESERVADO	`tests/unit/sales/test_reservation.py`
Gerar código e receber pagamento	`payments/application/payment.py` , gateway simulado, webhook HMAC	`tests/unit/payments/` ; sem integração bancária real
Retirar e dar baixa	ConfirmarRetirada, BaixarVeiculo, ConcluirVenda na ASL	Testes de SAGA; sem integração com órgão de trânsito
Concorrência de compradores	Escrita condicional e transação DynamoDB; lock por cliente	Teste concorrente com Moto
Ausência de pagamento	Timeout da espera → cancelamento e liberação	ASL e `test_saga.py`
Desistência em qualquer passo	Decisão persistida antes do callback; estorno se pago	Testes de corrida em `test_saga.py`
Retirada ausente	Timeout → ResolverRetirada	ASL e testes
Dados para pagamento e documentação, com proteção	Clientes internos, cifra de CPF/RG/CNH, acesso autorizado e auditado	Relatório de segurança
Arquitetura serverless/gerenciada com justificativas	Terraform, Lambdas, DynamoDB, Step Functions, Cognito, SQS/SNS	Desenho da arquitetura e diagramas
Serviços de segurança e razões	Cognito, IAM/boundary, SSM/KMS gerenciado, S3/CloudFront, HMAC/SigV4	Controles não implementados documentados com justificativa de custo
Relatório de segurança dos dados	Inventário, acesso, operação, riscos, limitações e retenção	data-security-report.pdf
Tipo de SAGA e justificativa	Orquestrada (Step Functions Standard)	saga-report.pdf